Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

CODEIGNITER PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 Q&A on MVC, routing, Query Builder, validation, and testing

TOTAL: 10 QUESTIONS

Q1 What is CodeIgniter and how does it implement MVC? Threat Model

Q2 How does CodeIgniter 4 routing work? Architecture

Q3 How does the CodeIgniter Query Builder prevent SQL injection? Configuration

Q4 What are CodeIgniter 4 Models and Entity classes? Hardening

Q5 How does CodeIgniter validation work? SSO / IdP

Q6 How does CodeIgniter 4 handle sessions differently from PHP native sessions? Compatibility

Q7 How do you handle file uploads in CodeIgniter 4? Lifecycle

Q8 What is a CodeIgniter RESTful Resource Controller? Compliance

Q9 How does CodeIgniter 4 Filters work? Testing

Q10 How do you test a CodeIgniter 4 controller? Tuning

ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 **CodeIgniter 4 is a lightweight PHP framework.** Mitigation

CodeIgniter 4 is a lightweight PHP framework. Controllers in app/Controllers extend BaseController and handle HTTP logic. Models in app/Models extend Model and interact with the database. Views in app/Views are PHP files rendered with return view('name', data).

A2 **Routes are defined in app/Config/Routes.php:** Primitives

Routes are defined in app/Config/Routes.php: $routes->get('users/(:num)', 'UserController::show/$1'). $routes->resource('users') creates CRUD routes automatically. Auto-routing (deprecated) mapped URLs to controllers by convention.

A3 **Query Builder methods accept raw values that** Hardening

Query Builder methods accept raw values that are automatically escaped and parameterized. $db->where('id', $id)->get('users') generates a prepared statement. Never concatenate user input into query strings; use Query Builder or $db->query('SELECT * FROM u WHERE id=?', [$id]).

A4 **A Model extending CI4's Model gets CRUD** Gotchas

A Model extending CI4's Model gets CRUD helpers (find, insert, update, delete) and built-in validation. An Entity class maps a database row to a typed PHP object with casting (int, bool, datetime). Set protected $returnType = UserEntity::class in the model.

A5 **Use $this->validate($rules) in a controller, where $rules** Hardening

Use $this->validate($rules) in a controller, where $rules is an array like ['email' => 'required|valid_email']. Define reusable rule sets in app/Config/Validation.php. $this->validator->getErrors() returns field-level error messages after a failed validation.

A6 **CI4's session library wraps native sessions but** Observability

CI4's session library wraps native sessions but adds configurable drivers (files, database, Redis, Memcached). Sessions are signed with an encryption key to prevent tampering. set_flashdata('key', val) stores one-request messages. Regeneration is automated.

A7 **Access via $file = $this->request->getFile('avatar'). Call** Automation

Access via $file = $this->request->getFile('avatar'). Call $file->isValid() and $file->hasMoved() before moving. $file->move(WRITEPATH.'uploads', $file->getRandomName()) stores it. Validate MIME type with getClientMimeType() and check file size against maxSize.

A8 **php artisan make:controller UserController --rest generates** Maintenance

php artisan make:controller UserController --rest generates index, show, create, store, edit, update, delete stubs. $routes->resource('users', ['controller'=>'UserController']) maps HTTP verbs to these methods. Override only the methods your resource needs.

A9 **Filters (like middleware) run before or after** Assessment

Filters (like middleware) run before or after controllers. Define a filter class with before() and after() methods. Register in app/Config/Filters.php under $aliases. Apply globally or per-route: $routes->get('admin/.*', 'AdminController', ['filter' => 'auth']).

A10 **Extend CIUnitTestCase. Use $result =** Optimization

Extend CIUnitTestCase. Use $result = $this->withUri('http://example.com/users/1')->controller(UserController::class)->execute('show', 1). $result->assertStatus(200) and $result->assertSee('Alice') check the response without starting an HTTP server.